



UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE
CIENCIAS EXACTAS E
INGENIERIAS

Seminario de traductores 2

Reporte de actividad final

Nombre del alumno: Mariana Soledad Román
Benítez
Profesor: Michel Emanuel López Franco
Fecha: 01 diciembre 2025

Etapa generación de código

Analizadores.py

```
analizadores.py X
analizadores.py > ...
1793
1794     self.tmp = ""
1795     self.continua = True
1796
1797     cad = " int sum(int a){\
1798         int z;\
1799         z = a + 2;\
1800         return z;\
1801     }\
1802     int main(){\
1803         int x;\
1804         int z;\
1805         x = 20;\
1806         z = 12;\
1807         z = sum(x);\
1808         print(z)\
1809         return z;\
1810     }"
1811     gr.txtE.insert(gr.END, cad)
1812
1813     divcad = cad.split()
```

Función sum: Se define una función llamada sum que toma un argumento a de tipo entero.

Dentro de la función, se declara una variable z de tipo entero, se le asigna el valor de a + 2, y luego se devuelve z.

Main.py

```
main_func.py X
main_func.py > reglas
1  import analizadores as an
2
3  def auxreglas():
4      n = 1
5      file = open('rgl.txt', 'r')
6      line = file.readlines()
7      for l in line:
8          l = l.rstrip()
9          an.auxregl.append(l.split('\t'))
10
11     for obj in an.auxregl:
12         obj = an.Regla(n, int(obj[0]), int(obj[1]), str(obj[2]))
13         n+=1
14         an.lisreglas.append(obj)
15     file.close()
16
17     def buscar(str):
18         for objlex in an.listalexico:
19             if objlex.cad == str:
20                 return objlex
21             else:
22                 pass
```

La función `auxreglas` lee un archivo llamado `'rgl.txt'` y procesa sus líneas, cada línea se limpia de espacios al final con `rstrip()`. Los resultados se almacenan en la lista `an.auxregl`.

Luego, cada elemento de `an.auxregl` se utiliza para crear un objeto de tipo `an.Regla` y se añade a la lista `an.lisreglas`. Se utiliza una variable `n` para asignar un número único a cada regla.

La función `buscar` busca un objeto en la lista `an.listalexico` cuyo atributo `cad` sea igual al parámetro de entrada `str`. Si encuentra un objeto que cumple la condición, lo devuelve. Si no, no hace nada.

```
24 def reglas():
25     file = open('compilador.lr', 'r')
26     line = file.readlines()
27     for l in line:
28         l = l.rstrip()
29         an.matrizreglas.append(l.split('\t'))
30
31     for i in range(len(an.matrizreglas)):
32         for j in range(len(an.matrizreglas[i])):
33             an.matrizreglas[i][j] = int(an.matrizreglas[i][j])
34     file.close()
35
36 def eliminalistaVar(self):
37     an.an.pila.pop()
38     an.pila.pop()
39     an.pila.pop()
40     self.data = an.pila.pop()
41     an.pila.pop()
42     an.pila.pop()
43     an.listavar.append(an.DefVar('Unknown ', self.data, self.lv))
```

La función `reglas` realiza una operación similar a `auxreglas` pero con un archivo llamado `'compilador.lr'` y los resultados se almacenan en `an.matrizreglas`. Luego, se convierten los elementos de la matriz a enteros.

El método manipula pilas (`an.an.pila` y `an.pila`) y realiza operaciones de `pop` en ellas.

Luego, crea un objeto `an.DefVar` y lo agrega a la lista `an.listavar`.

Compilar

Analizador Léxico

```
int sum(int a){    int z;    z = a + 2;
    return z;    }    int main(){    int
x;    int z;    x = 20;    z = 12;    z
= sum(x);    print(z)    return z;    }
```

Entrada

Lexema	Token	Tipo
int	Tipo	4
sum	Identificador	0
(	Parentesis	14
int	Tipo	4
a	Identificador	0
)	Parentesis	15
{	Corchete	16
int	Tipo	4
z	Identificador	0
;	Punto y coma	12

Analisis Léxico

Analizador Semántico

Tipo	Identificador	Ámbito
int	z	sum
int	x	main
int	z	main

Analisis Semántico

Analizador Sintáctico

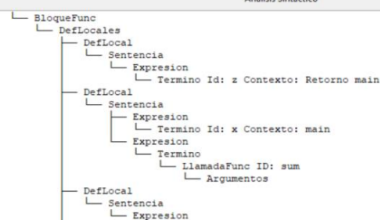
```
g0
p0int5
p0int5sum0
p0int5sum0(11
p0int5sum0(11int15
p0int5sum0(11int15a18
p0int5sum0(11int15a18ListaParam21
p0int5sum0(11Parametros14
p0int5sum0(11Parametros14)17
p0int5sum0(11Parametros14)17(20
p0int5sum0(11Parametros14)17(20int5
p0int5sum0(11Parametros14)17(20int5e8
p0int5sum0(11Parametros14)17(20int5e8ListaVar9
p0int5sum0(11Parametros14)17(20int5e8ListaVar9;12
p0int5sum0(11Parametros14)17(20DefVar25
p0int5sum0(11Parametros14)17(20DefLocal24
p0int5sum0(11Parametros14)17(20DefLocal24z27
p0int5sum0(11Parametros14)17(20DefLocal24z27=35
p0int5sum0(11Parametros14)17(20DefLocal24z27=35a46
p0int5sum0(11Parametros14)17(20DefLocal24z27=35Termino44
```

```
r30: ValorR
d57
r24: Senten
r19: DefLoc
r15: DefLoc
r16: DefLoc
r16: DefLoc
r16: DefLoc
r16: DefLoc
r16: DefLoc
d33
r14: BloqFu
r9: DefFunc
r5: Definic
r2: Definic
r3: Definic
r3: Definic
r1: program
r0: accept
```

Analisis Sintáctico

Salida

Árbol ---->



Ensamblador.asm

Código de ensamblador generado

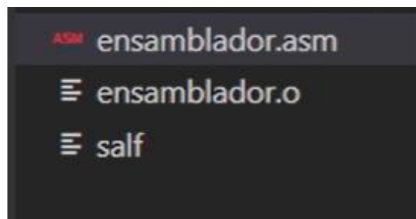
```
ASM ensamblador.asm X
ASM ensamblador.asm
1  section .data
2      primr: db "La impresion es := %lf",10,0
3  section .bss
4      resp: resq 2
5  section .text
6
7  extern printf
8  global sum
9  , main
10 sum:
11      PUSH rbp
12      MOV rbp, rsp
13      SUB rsp, 48
14      MOV QWORD [rbp -24], rdi
15      MOV rax, QWORD [rbp -24]
16      MOV rdi, 2
17      ADD rax, rdi
18
19      ADD rsp, 48
20      MOV rsp, rbp
21      POP rbp
22      ret
```

```
24  main:
25      PUSH rbp
26      MOV rbp, rsp
27      SUB rsp, 48
28      MOV WORD [rbp -4] , 20
29      MOV WORD [rbp -8] , 12
30      MOV rax, QWORD [rbp -4]
31      MOV rdi, rax
32
33      call sum
34      MOV QWORD [rbp -8], rax
35
36      PUSH qword[rbp -8]
37      FILD dword[rsp]
38      FSTP qword[rel resp]
39      ADD rsp, 8
40      MOVSD xmm0,qword[rel resp]
41      MOV rdi, primr
42      MOV al, 1
43      call printf WRT ..plt
44      MOV rax, QWORD [rbp -8]
45
```

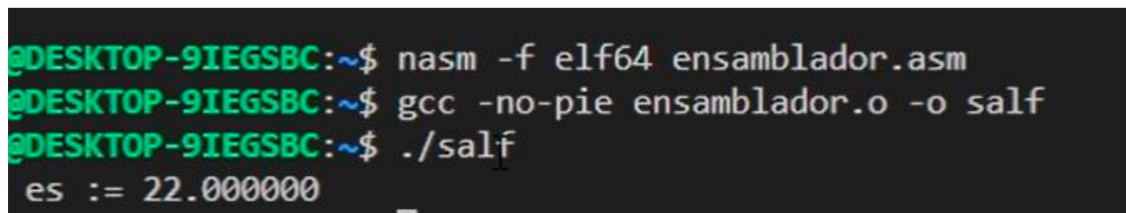
```
46      ADD rsp, 48
47      MOV rsp, rbp
48      MOV rax, 60
49      MOV rdi, 0
50      syscall
51
```

Procedimiento

Hay que copiar el Código generado en el ensamblador y pegarlo en otra ventana con la distribución de Ubuntu para poder utilizar el ensamblador.



- **nasm -f elf64 ensamblador.asm:** Nos va a crear el archivo ensamblador.o
- **gcc -no-pie ensamblador.o -o salf:** El segundo lo compila
- **./salf:** Y por último nos metemos a la carpeta en la que está nuestro Código



Al ser la impresión 22 podemos notar que coincide, entendiendo lo anterior explicado sobre la función suma.